Backend (.NET Core) Developer Training Plan

Training Overview 2
Phase 1: Fundamentals of Git (Practical Required) 2
Phase 2: C# Language & SOLID Principles 2
2.1 Introduction to C# 3
2.2 Classes and OOP Concepts 3
2.3 Types & Assemblies 3
2.4 Members: Methods, Events & Properties 3
2.5 Flow Control 3
2.6 Exception Handling 4
2.7 Collections & Generics 4
2.8 SOLID Principles 4
Phase 3: SQL & PostgreSQL Basics 4
Phase 4: Data Access & ORM 5
4.1 ADO.NET 5
4.2 LINQ 5
4.3 Entity Framework Core 5
4.4 Dapper 5
Phase 5: ASP.NET MVC & Web API 5
5.1 MVC(Overview Only) 6
5.2 ASP.NET Web API 6
Phase 6: .NET Core 6
6.1 .NET Core 6
6.2 Duende Identity Server 7
Phase 7: Security 7
Phase 8: Design Patterns 7
Phase 9: Latest Enhancements 8
Phase 10: Azure DevOps & Cloud 8
Azure branching policy 8
HRMS Proof of Concept (POC) Application Development 9

Training Overview

This learning path is designed to build strong fundamentals in backend development using **Microsoft .NET Core with C#**, secure identity management, multitenant architecture, database development, and deployment practices. The trainee will also build a **HRMS POC Application** as hands-on project work.

- **Important items in the roadmap are marked in red.**

Phase 1: Fundamentals of Git (Practical Required)

Topics

- What is Git and GitHub
- Local vs Remote repositories
- Basic Git commands (init, clone, commit, pull, push)
- Branching strategy and types of branches
- Merge, conflict handling, rebase, reset, revert
- Cherry-pick

Reference:

https://www.youtube.com/watch?v=CvUiKWv2- HYPERLINK "https://www.youtube.com/watch?v=CvUiKWv2-C0" HYPERLINK "https://www.youtube.com/watch?v=CvUiKWv2-C0" HYPERLINK "https://www.youtube.com/watch?v=CvUiKWv2-C0"C0

Phase 2: C# Language & SOLID Principles

2.1 Introduction to C#

- What is .NET, CLR, FCL
- Compilation flow in C#
- Debugging in Visual Studio
- Data types, variables, coding standards

2.2 Classes and OOP Concepts

- Types of classes
- Constructors and object creation
- Reference types
- Access modifiers
- Static members & Singleton
- Abstract classes and Interfaces

2.3 Types & Assemblies

- Value vs Reference types
- Structs, Enums, Records
- Immutability
- Nullable types
- Assemblies & Namespaces

2.4 Members: Methods, Events & Properties

- Methods and types
- Properties, fields
- Delegates and events
- Extension methods

2.5 Flow Control

- Conditions

- Loops
- Jump statements

2.6 Exception Handling

- Types of exceptions
- Try/Catch/Finally
- Custom exception handling
- `using` keyword

2.7 Collections & Generics

- Collection types
- Generic programming

2.8 SOLID Principles

Reference:

› https://dotnettutorials.net/

› https://www.c- HYPERLINK "https://www.c-sharpcorner.com/" HYPERLINK "https://www.c-sharpcorner.com/" HYPERLINK "https://www.c-sharpcorner.com/"sharpcorner.com/

› YouTube: C# HYPERLINK "https://www.youtube.com/results?search_query=c%23+tutorials" HYPERLINK "https://www.youtube.com/results?search_query=c%23+tutorials" HYPERLINK "https://www.youtube.com/results?search_query=c%23+tutorials"Tutorials

Phase 3: SQL & PostgreSQL Basics

Topics

- DDL, DML, DCL, TCL
- ACID Properties
- Filtering, Sorting, Grouping & Paging
- Joins, Subqueries, Apply
- Temp tables, CTE, Case/When
- Views & Indexing
- Stored procedures, functions, triggers
- Cursors, sequences

- SQL profiling and execution plans

Reference:

› https://www.sqlservertutorial.net/

Phase 4: Data Access & ORM

4.1 ADO.NET

- Connections, commands, data readers
- Connected vs disconnected architecture
- Calling stored procedures

4.2 LINQ

- Query syntax & method syntax
- Lambda expressions
- Deferred vs Immediate execution

4.3 Entity Framework Core

- What is EF Core? Pros & cons
- Code-First & Database-First approaches
- Migrations and seeding
- Lazy vs eager loading
- Async queries
- Calling stored procedures

4.4 Dapper

- Query, Insert, Update using Dapper
- Stored procedures in Dapper
- Dynamic parameter
- Bulk operations

Reference:

› https://www.learndapper.com/

Phase 5: ASP.NET MVC & Web API

5.1 MVC(Overview Only)

- MVC architecture and lifecycle
- Routing basics
- Controllers & action results
- Filters
- Views & partial views (conceptual only)
- MVC is covered **only to understand architectural concepts**
- **No MVC-based projects or practical implementation required**
- All hands-on work must be done using **ASP.NET Core Web API**

5.2 ASP.NET Web API

- REST vs SOAP
- HTTP verbs, response formats
- API versioning
- Authentication & Authorization (Role-based, Token based, Policy based)
- Token-based security
- OpenID, OAuth
- Caching & localization
- API health check monitoring
- Swagger, Postman testing

Phase 6: .NET Core

6.1 .NET Core

- .NET Framework vs .NET Core
- Environment configuration
- Middleware pipeline
- DI Lifecycle: Singleton, Scoped, Transient

- Identity Core authentication
- Asynchronous programming (`async/await`)
- Exception handling & global error handling
- Logging (Serilog)
- Deployment on IIS

Reference:

https://www.youtube.com/watch?v=4IgC2Q5- HYPERLINK "https://www.youtube.com/watch?v=4IgC2Q5-yDE" HYPERLINK "https://www.youtube.com/watch?v=4IgC2Q5-yDE" HYPERLINK "https://www.youtube.com/watch?v=4IgC2Q5-yDE"yDE

6.2 Duende Identity Server

- What is Duende IdentityServer ?
- Duende IdentityServer Architecture
- Duende IdentityServer Configuration
- Duende IdentityServer with ASP.NET Core Identity
- Duende IdentityServer Token Management

Reference:

https://www.youtube.com/watch?v=eTyL89DoDfw

Phase 7: Security

Topics:

- SQL Injection defense
- Anti-forgery tokens
- DoS & DDoS awareness

Phase 8: Design Patterns

Topics:

- Singleton

- Repository
- Factory & Abstract Factory
- Mediator
- CQRS
- Other common enterprise design patterns

Phase 9: Latest Enhancements

- C# 12 / 14 features
- .NET 8 / .NET 10 updates

Phase 10: Azure DevOps & Cloud

- CI/CD pipeline creation
- Repo/Branch strategy
- Deployment automation

Reference:

https://learn.microsoft.com/en- HYPERLINK "https://learn.microsoft.com/en-us/azure/devops/repos/git/gitquickstart" HYPERLINK "https://learn.microsoft.com/en-us/azure/devops/repos/git/gitquickstart" HYPERLINK "https://learn.microsoft.com/en-us/azure/devops/repos/git/gitquickstart"us/azure/devops/repos/git/gitquickstart

How To Use AI Tools:

- Visit https://www.deeplearning.ai/ and sign up using your eVision email address.
- Refer to the course tutorial: Claude Code HYPERLINK "https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant"– HYPERLINK "https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant" A Highly Agentic Coding HYPERLINK "https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant" HYPERLINK "https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant" HYPERLINK "https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant"Assistant

NOTE :

If the link redirection shows a "This website is blocked" page as shown below,please contact the Network and Administrator team to request access.



Azure branching policy

The link of the below document will let us provide the branch strategy we are using to commit, merge and deploy code as per the different environemetns

https://wiseliy.sharepoint.com/:w:/r/sites/WISELIY/_layouts/15/Doc.aspx?sourcedoc=%7B96A4216C-3D69-487C-B97D-F3CBC0BA54D6%7D&file=Wiseliy%20Repository%20Branching%20%26%20Version%20Control%20Guidelines.docx&action=default&mobileredirect=true

HRMS Proof of Concept (POC) Application Development

The goal of this POC is to develop a simple Human Resource Management System where organizations can register and manage employees. The application will demonstrate essential features such as authentication, role-based access, CRUD operations, and API documentation.

Core Features

- **User Registration & Authentication**
  - Implement authentication using **ASP.NET Core Identity Framework**.
  - Support secure **user registration and login**.
  - Password hashing, validations, and secure JWT token handling.
- **Organization Management**
  - Allow new users to create an **Organization** after registration.
  - Required organization details:
    - Name
    - Address
- **Role-Based Access**
  - system roles:
    - **Admin**

- **HR**

Role Permissions:

- **Admin**
  - Can add or remove HR users.
  - Can view all employees, including HR created under the organization.
- **HR**
  - Can create, edit, and delete **only the employees created by themselves**.

- **Employee Management**
- HR can manage employee records with the following fields:
  - First Name
  - Last Name
  - Email
  - Phone Number
  - Address

Technical Requirements

- **Backend Framework:** ASP.NET Core Web API
- **Authentication:** ASP.NET Core Identity Framework
- **Database:** PostgreSQL
- **ORM Options:**
  - **Dapper** or **Entity Framework Core** (use one or both for demonstration)
- **API Documentation:** Swagger
- **Validation:**
  - Use Fluent Validation or Data Annotations for input validation
- **Repository Pattern & Service Layer**
  - Use clean architecture principles for separation of concerns

Source Control & Deployment

- Use **Git** for version control.
- Push all source code to a dedicated **Git repository** (GitHub).

Multi-Tenancy Architecture with .NET Core and PostgreSQL

https://wiseliy-my.sharepoint.com/:w:/r/personal/milannayak_evisionits_com/_layouts/15/doc2.aspx?sourcedoc=%7B8626AAA4-BA62-4FB6-AAC1-1C62906C79F3%7D&file=Document%201.docx&action=editNew&mobileredirect=true&wdOrigin=DOCUMENTS.SHELL%2CAPPHOME-WEB.BANNER.NEWBLANK&wdPreviousSession=2d6d5db9-a1ee-4c89-91a3-063d6644f808&wdPreviousSessionSrc=AppHomeWeb&ct=1790577255275